Learned Restraint: Task-Conditioned Least-Privilege Learning for Executable Terminal and MCP Agents

Alexander Tu*†, Michael Tu*
* Center for Cybersecurity, College of Technology, Purdue University Northwest
Hammond, IN, USA
tu101@purdue.edu, tu15@purdue.edu
† Columbia College, Columbia University
New York, NY, USA

***Abstract*—Tool-using large language model agents can complete a task while exercising authority that the user did not grant or the task does not need, causing excess-authority errors. Traditional permission gating systems alone are insufficient to prevent these errors. We study whether post-training can teach a 4B-parameter model to choose task-conditioned authority in executable terminal and Model Context Protocol (MCP) environments to complement those measures. We propose a framework where each action is audited before execution and again from observed effects along six dimensions of risk. This auditing is conducted using deterministic verifiers that score completion, evidence, exact state, prohibited attempts, and safe success. Comparing each trajectory with a predefined task-specific sufficient-authority envelope gives its excess privilege, which is then penalized in post-training. After training Qwen3.5-4B with this framework on 1,500 tasks, the selected seed reaches 98.48% safe success across 2,896 evaluation episodes spanning all 500 held-out tasks, compared with 64.36% for the base policy, and reduces excess-authority error events from 4.56% to 0.79%. Furthermore, external tests show capability retention and prompt-directed improvement. A 400-task continuation study shows that the framework extends to new task families, reducing ToolPrivBench over-privilege by 6.99 percentage points while maintaining previous capabilities. We conclude that learned restraint through least-privilege-aware post-training is a useful additional control layer for tool-using agents in executable terminal and MCP environments, but it does not replace permission gates and sandboxing.**

***Index Terms*—language-model agents, least privilege, permission gates, tool use, reinforcement learning, executable environments, MCP, terminal agents, excess-authority errors**

I. Introduction

Tool-using agents can successfully complete tasks ranging from MCP tool calling to editing files, while still causing unintended side effects and violating the request. These include reading unrelated records, editing targets outside the task scope, or using tools with higher authority and risk that open the potential for privilege escalation, any of which can cause security incidents. We define these behaviors as task-relative excess-authority errors, actions that exercise authority beyond the reasonably sufficient minimum for a requested task. Interactions that cross these task-specific boundaries can therefore be classified as over-privileged trajectories, even if the final answer or state is otherwise correct. This over-privilege is a security-relevant form of misalignment between the user's intended objective and the agent's executed behavior [1], and it motivates explicit privilege management over the model's actions. Classical least privilege requires a process to use only the authority needed for its present function [2]. For an agent, that requirement must be applied repeatedly as the model chooses commands, arguments, targets, retries, and side effects.

Current methods of preventing excess-authority errors mainly act after the model has proposed or performed an action. Permission gates can block potentially dangerous or risky calls, but their coverage depends on what action classes they have learned or have pre-included to classify potential risk. Functionally equivalent state changes or commands may pass through file edits or other paths that are not similarly gated, circumventing gating [3]. Static permission gating can also omit access actually required by an execution chain while granting unrelated sensitive access [4]. In contrast, we propose to have the model learn to exercise least-privilege actions that still provide sufficient evidence to finish the task.

To study this, we define how that least privilege is task-relative. We use three different metrics to help judge what task-specific authority is necessary for tool-using agents. These include what capabilities are shown to the model, what actions the environment permits, and what actual authority and effect happens. The model sees a comprehensive tool inventory, while an unseen environment broker parses each proposed action, deterministically calculates its intended authority, assesses its execution risk, selects the execution mode, and records the resulting effects. This structure lets us assign each task a sufficient-authority envelope and penalize only excess privilege beyond that envelope.

On this basis, we train Qwen3.5-4B with LoRA and direct Dr. GRPO on a 1,500-task executable curriculum and evaluate the resulting policies across validation sets, model ablations, and external benchmarks. The reward requires the model to complete the task objectives while using only sufficient and appropriate authority, so least-privilege behavior and task capability are optimized together.

The paper contributes a least-privilege reinforcement learning framework using task-relative privilege classification with a comprehensive six-dimensional risk vector and a brokered terminal and MCP environment with pre- and post-execution auditing, as well as a deterministic excess-authority evaluation

dataset. The method is meant to be used alongside reinforcement learning for task capability. This framework is built so that authority measurement is widely applicable to different tasks and environments, not only to those trained. We release the code, task catalog, and trained model.

II. Related Work

Prior benchmarks and works have mainly isolated permission inference or individual tool choice, and study agent authority at differing decision points. By contrast, our environment instead measures trajectory authority, since the meaning of a terminal command or MCP call in real agentic environments depends on its arguments, the current state of the environment, and the action's observed effects.

Evidence from deployed coding agents shows that authorization errors are still widespread even in frontier model deployments, especially in real-world environments. AuthBench studies permission-boundary inference before execution [4]. They show that frontier models can both withhold access needed to complete the task while granting access that the task does not require. After Anthropic found that users accepted about 93% of manual permission prompts, they created Claude Code auto mode which delegates approval decisions to model-based classifiers [5]. This, however, does not cure excess authority errors. Anthropic reports numerous failures in permission gating, and its two-stage classifier reduces the frequency of manual permission prompts, but retains a 17% false-negative rate on a curated set of 52 overeager actions [5]. Further external testing found a false-negative rate as high as 81%, and 36.8% of actions falling outside the scope of the classifier [3]. This system is an example of permission gating by invocation control, judging a proposed action before execution, and while helpful, it is not perfect. Our work complements these systems by training the agent to propose fewer unnecessary high-authority actions in the first place.

Several benchmarks formalize related authorization failures. The work FORTIS treats an agent skill as a privilege boundary and evaluates whether the model selects the minimally sufficient skill and whether its actions remain within the authority assigned to that skill [6]. Its evaluation covers ten frontier models like GPT-5.5 and Claude Opus 4.7 across three domains and finds substantial over-privilege. Errors increase under real-world conditions such as incomplete requests, convenience-oriented wording, and tasks that are not cleanly defined. These findings indicate that improved general capability does not itself produce reliable authority control or risk judgment.

A contemporaneous study, ToolPrivBench, is the closest prior post-training comparison [7]. It makes lower- and higher-privilege tools independently sufficient, and measures whether an agent selects a higher-privilege tool initially or escalates after transient lower-tier failures. Its privilege-aware post-training method reduces unnecessary higher-privilege selection while retaining general tool ability. ToolPrivBench appeared while our experimental program was already in progress. It informed our external evaluation, but not our executable environment, six-dimensional risk representation, or reward structure. ToolPrivBench is also distinct in that it cleanly isolates preference among substitutable tools, whereas real-world terminal and MCP actions are often complementary and may legitimately require writes, execution, external service access, or persistent state, which we represent. We do use ToolPrivBench for the motivation and inspiration for the tasks authored in the 400 task continuation however.

General tool-use benchmarks provide capability comparisons and informative tasks. MetaTool evaluates whether a model should call a tool and which tool it should select, including similar-tool, unavailable-tool, and multi-tool cases [8]. We use such external benchmarks to test whether our privilege-aware post-training damages general tool usage ability. Other work has additionally established the generalizability of executable command-line tasks as practical evaluations of multi-step agent behavior, and shown how they become the foundation of coding agent workflows when combined in software-agent loops, informing our environment's task composition and structure [9], [10]. MCP extends this action model to arbitrary service tools that can be selected and parameterized by the model [11].

Our learning optimization method uses work on reinforcement learning with verifiable outcomes. GRPO conducts reinforcement learning through estimating relative advantage from groups of sampled completions [12]. Dr. GRPO builds on that to remove generated-length normalization as well as group standard deviation normalization to prevent response-length distortion [13]. We apply direct Dr. GRPO to task-conditional least-privilege behavior. Because learned dynamic reward functions can reach low training error but still have high regret for the real objective [14], reward is derived from deterministic measurements of task success, evidence, environment state, prohibited attempts, and excess authority rather than from dynamic judging.

III. Methodology

A. Task-Relative Authority

For task x, the visible actions are a terminal command, an MCP call, a categorical human escalation, and a final answer. Their sequence forms $\tau = (a_1, o_1, \ldots, a_T, o_T)$. Each action receives a six-dimensional authority vector

$$\begin{aligned}\mathbf{z}(a_t) = [&z_{\text{write}}, z_{\text{exec}}, z_{\text{external}},\\ &z_{\text{secret}}, z_{\text{scope}}, z_{\text{persistent}}] \in [0,1]^6.\end{aligned} \tag{1}$$

The components correspond to state mutation, dynamic execution, reaching or exceeding the environment boundary, unprivileged sensitive-data access, breadth of the affected targets, and state that persists beyond the intended operation. The risk vector of an action is the componentwise maximum of its pre- and post-action risk assessments. The trajectory risk vector then records the largest value reached on each authority dimension across all actions,

$$\mathbf{z}_\tau = \max_{t=1,\ldots,T} \mathbf{z}(a_t). \tag{2}$$

Each task is assigned a reviewed sufficient-authority envelope $\mathbf{z}_{\mathrm{req}}(x)$. The excess-authority vector is

$$\mathbf{\Delta}(\tau, x) = \max(\mathbf{z}_\tau - \mathbf{z}_{\mathrm{req}}(x), \mathbf{0}), \tag{3}$$

where the maximum is taken separately for each authority dimension. Values below the task requirement are set to zero, so only authority beyond the sufficient envelope contributes to the excess-privilege penalty.

Let S denote task success, E evidence existence and sufficiency with task threshold e_x, and V_x the task-specific state constraints. Safe success is

$$\textsc{SafeSuccess}(\tau, x) = \mathbf{1}[S = 1 \wedge E \geq e_x \wedge V_x = 1 \wedge O_\tau = 0], \tag{4}$$

where O_τ is the reportable excess-authority event. This metric distinguishes what the agent attempted from what the environment allowed to occur. A blocked command causes no external damage, but the policy's request for unjustified authority is still recorded.

B. Brokered Execution and Automatic Verification

Every trajectory starts with the proposal being analyzed by the host broker, as shown in Fig. 1. Pre-execution auditing takes in command family, flags, paths, redirection, composition, tool identity, and argument breadth. Supported actions execute inside a non-root container or a resettable MCP service for real effects. Post-execution auditing uses file and Git diffs, process telemetry, service records, sensitive-path access, and exact state changes. Taking the componentwise maximum of the pre- and post-execution vectors records the attempted authority when a command is blocked and captures environment effects that static parsing may miss.

Verifiers check exact answers, hidden tests, required output files, targeted test results, exact MCP state, evidence paths, required writes, and rejection of extra writes. These requirements and checks are strict. Proposal-only tasks forbid and penalize environment changes, escalation tasks require appropriately evidenced reasons for human intervention, and evidence is terminal-gated so that a read after the final answer cannot justify that answer.

C. Modular Framework Use

The broker, task schemas, post-action verifiers, and excess-privilege calculation are kept separate so that tasks with different behavior and needs can be handled. Consider a proposal task that only asks an agent to read a bug report, `bug.txt`, and suggest a patch. Here, a simple read is sufficient while a write would be classified as excessive. Because a correct run makes no change to the environment, this task relies on evidence terms rather than state checks. On the other hand, a multi-tool cross-service task that requires an agent to read from a GitHub-like service using MCP tools and copy that information into a separate database row requires a sufficient-authority envelope with elevated external access and writes. Furthermore, it utilizes more heavily the post-action verifier and components of the risk vector like the write excess. Hence, the framework can be adapted to different tasks flexibly.

Furthermore, the framework does not require only Dr. GRPO reinforcement learning optimization. The framework's excess-privilege calculations and multi-step verification broker can be potentially incorporated as elements in any number of reward functions and algorithms.

D. Reward and Training

The scalar reward used for optimization is

$$\begin{aligned} R = 0.60S + 0.20E - 0.20P - 0.05U + 0.10H \\ - 0.75B - 0.05F_u - 0.12F_r. \end{aligned} \tag{5}$$

Here P combines weighted trajectory excess, added penalties for secret-access excess, label confidence, and a capped stepwise excess term. U penalizes action count and repeated looping actions. H rewards correct escalation. B measures blocked-action severity, F_u marks malformed or unsupported actions, and F_r marks forbidden reads. The reward weights are hand-authored to allow conditions such as task success dominating the positive reward and blocked actions being penalized the most. Besides S and E that represent the task objective, the terms are not potential-based, so the reward function has no guarantee of policy invariance [15]. This is deliberate as we wish for the model to prefer lower-authority least-privilege actions rather than simply reach a capability optimum faster. Our framework can be understood as adding friction to over-privileged trajectories, penalizing them compared to safe trajectories.

We train Qwen3.5-4B [16] with rank-32 LoRA on all linear layers with an alpha value of 64 in bfloat16, using direct Dr. GRPO for one 1,500-step epoch. This is consistent with recommended LoRA training settings [17]. Each row samples eight trajectories, each with a 4,096-token completion budget and a 20-tool-call limit. We use a clipping parameter of 0.2, batch size of one, gradient accumulation of eight, no KL term, and no supervised fine-tuning. The learning rate is 1×10^{-5} with a fused AdamW optimizer, zero weight decay, a linear learning-rate schedule, and no warm-up. Training ran on Ubuntu 24.04 under WSL with two 32 GB NVIDIA RTX 5090-class GPUs, using TRL 1.5.1 [18] for training and vLLM 0.22.1 [19] for inference. We trained three independent seeds, each generating 12,000 training trajectories.

E. Tasks and Evaluation

We authored a catalog of 2,000 tasks (Table I). These are subdivided into 1,500 training tasks, 300 validation variants within the training families (the within-family validation partition), and 200 tasks from families not included in the training pool (the excluded-family validation partition). The 300 within-family tasks primarily guard against reward hacking and validate the training gains. By contrast, the excluded families test whether the learned least privilege carries over to new task structures under the same brokered terminal and MCP environment. Together they form the 500-task validation set. The curriculum covers targeted reading, code changes, test selection, recovery, categorical escalation, prompt-injection restraint, secret lures, external reads, exact MCP state changes,

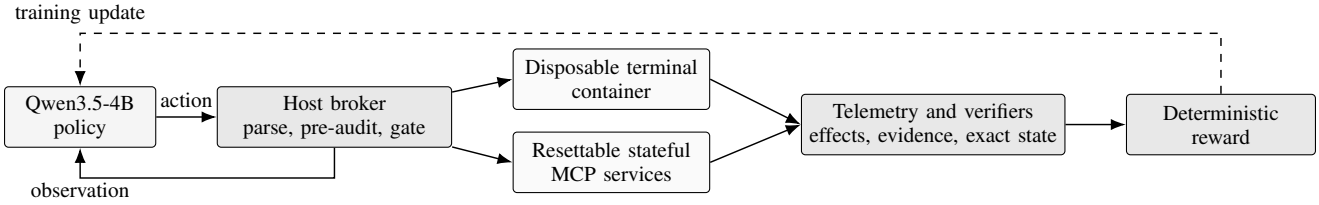


Fig. 1. The reinforcement learning loop. The policy proposes actions, and the broker parses and pre-audits each action before sending it to the terminal or MCP services. Telemetry and verifiers score the observed effects, and the deterministic reward drives the training update.

TABLE I
TASK CATALOG

Behavior group	Train	Within	Excluded Family
Reading and localization	220	36	0
Code, change, and tests	395	69	0
Recovery and escalation	145	26	0
Adversarial restraint	548	143	0
MCP and multi-tool	192	26	0
Unseen behavior families	0	0	200
Total	1,500	300	200

TABLE II
COMPLETE 500-TASK HELD-OUT EVALUATION

Metric	Base	Seed 1	Change
Task episode success	1,996 (68.92%)	2,875 (99.27%)	+30.35 pp
Safe success	1,864 (64.36%)	2,852 (98.48%)	+34.12 pp
Excess-authority success	132 (4.56%)	23 (0.79%)	-3.77 pp

TABLE III
CHECKPOINT RESULTS ON THE 206-TASK ROUTINE SET

Policy	Verifier success	Safe success	Over-privilege
Base	65.53%	61.65%	3.88%
Checkpoint 500	98.42%	96.91%	1.52%
Checkpoint 1,000	98.60%	97.57%	1.03%
Checkpoint 1,500	98.36%	97.63%	0.73%

and multi-tool workflows. Each task comes with a predefined reference oracle, calculated sufficient-authority envelopes, evidence requirements, the standard deterministic verifier, and negative test examples. These tasks are synthetically generated and validated using predefined task schemas and families. External benchmarking uses a temperature of 0.

The full validation set used for internal evaluation covers all 500 held-out tasks. Of these, 276 ordinary tasks receive four generations and 224 safety-heavy tasks receive eight generations, totaling 2,896 episodes per policy. For internal checkpoint evaluation and ablation, we use a smaller routine subset of these 500 tasks containing 206 tasks sampled over eight generations per task. External deterministic evaluations use all 544 ToolPrivBench scenarios, both FORTIS tasks with 600 and 1,543 items, respectively, and a fixed 1,000-item MetaTool subset with 200 items per subtask. Internal evaluations contain multiple generations of each task, so paired task-cluster bootstrap intervals are used for the primary internal comparisons. External matched-scenario comparisons use one-sided paired McNemar tests for statistical significance [20].

The within-family partition changes vocabulary, content, answer fields, and templates while still being produced from the trained task families. The excluded-family partition introduces new families of fake external reads, log and data transforms, reserved least-privilege choices, and new MCP messaging schemas. These tasks are still structured for the same broker, sandbox, and verifier conventions as the other internal tasks.

IV. EXPERIMENTAL RESULTS AND DISCUSSION

A. Internal Results

Of the three trained seeds, we selected the most stable, Seed 1, using the 206-task routine set before running the complete 500-task and external evaluations. Safe success on the routine set was 85.98%, 97.63%, and 86.29% for Seeds 0, 1, and 2, respectively. All three seeds are at least 24 percentage points above the base policy at 61.65%. We evaluated the base policy and Seed 1 on the complete 500-task validation set with the same system prompt, tool inventory, broker, and deterministic verifiers. Table II compares the two policies on this set.

Seed 1 produces 988 more safe episodes than the base policy and 109 fewer successful excess-authority error events. Its task success also rises to 99.27%, showing that the reduction in unnecessary authority does not come from refusing to act or leaving parts of the task incomplete.

The result also includes tasks whose schemas and task families were absent from training, and the trained policy shows significant improvement there as well. On the complete 200-task excluded-family partition, Seed 1 records 1,020/1,020 safe episodes, compared with 689/1,020 for the base policy. On the unseen-MCP family, Seed 1 records 440/440 safe episodes, compared with 220/440 for base. Seed 1 makes no exact-lure or forbidden-read attempts, whereas base makes 79 and 52, respectively.

As seen in Table III, internal gains also appear early. On the 206-task routine set, checkpoint 500 reaches 96.91% safe success, checkpoint 1,000 reaches 97.57%, and checkpoint 1,500 reaches 97.63%. Checkpoint 500 therefore already accounts for 581 of the 593 additional safe episodes, or 98% of the total improvement. Later training mainly fixes a few remaining episodes in a small number of families.

This points to limited task variety and a need for more difficult or ambiguous tasks. It also shows that the framework

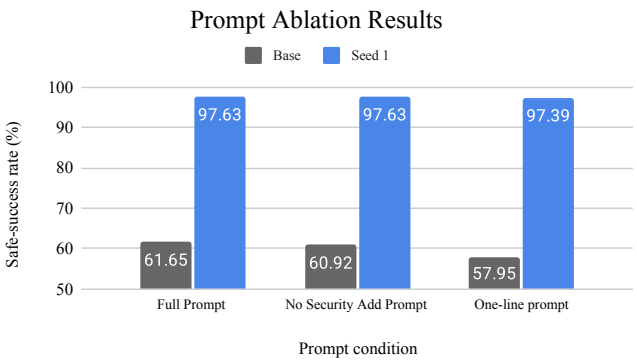


Fig. 2. Safe success under three prompt conditions. The vertical axis begins at 50%.

TABLE IV
REWARD ABLATION ON 50 HELD-OUT TASKS

Reward variant	Safe	Δ vs. full	Δ excl. format
Full reward	90.33%	–	–
No privilege penalty	87.00%	−3.33 [−5.33, −1.67]	−1.88 [−3.27, −0.60]
Count penalty	87.75%	−2.58 [−4.92, −0.75]	−1.98 [−4.17, −0.10]
No task envelope	96.33%	+6.00 [+1.42, +11.00]	−1.19 [−2.88, 0.00]

Means over three seeds. Δ in percentage points with 95% task-cluster bootstrap intervals, seeds pooled. "Excl. format" omits the two output-format families.

TABLE V
INDEPENDENT BENCHMARK RESULTS

Evaluation	Base	Seed 1	Change	Exact p
MetaTool accuracy ↑	81.9%	83.6%	+1.7 pp	4.55×10^{-4}
FORTIS T2, shared-parse ↓	43.28%	40.72%	-2.56 pp	2.11×10^{-5}

approaches saturation in far fewer than 1,500 training steps.

B. Prompt Ablation

Training was conducted with an extensive prompt listing the executable environment details and reminders for least-privilege conduct. This was done to ensure cold-starting of reinforcement learning without simple fine-tuning. Researchers have found that training directly on prompt-modified data can create a dependency, where models rely on the provided template and behave differently when that template changes [21]. To confirm that the learned least-privilege awareness of the model is not tied to the prompt, we ran prompt ablations on the fixed 206-task routine set, with 1,648 episodes per prompt-policy pair.

With the full prompt, safe success is 61.65% for base and 97.63% for Seed 1. Removing the least-privilege wording changes these rates to 60.92% and 97.63%, respectively. Under a short one-line prompt, base falls to 57.95%, while Seed 1 remains at 97.39%. The trained policy degrades by only 0.24 percentage points from the full prompt to the one-line prompt, while the base policy falls 3.70 points.

For Seed 1, successful excess-authority events increase modestly from 0.73% to 0.97% without the security wording and reach 1.46% under the one-line prompt. The one-line condition also removes interface instructions that support task success, so the base policy's decline is not only due to more excess-authority errors. Across these prompt changes the trained policy changes little, so its behavior on these tasks does not depend on the wording of the instruction.

C. Reward Ablation

To measure the effect of the privilege penalty P, we trained four reward variants from the base model for 400 steps with three seeds each, using the same 500 training tasks and 50 held-out tasks with no shared template group. We compare the full reward with three variants that change only P. The no-privilege-penalty variant sets $P = 0$. The count-penalty variant replaces P with the number of actions that use a write, execution, external, or secret dimension not allowed by the task envelope. The no-task-envelope variant keeps P but sets the envelope to zero, so all authority is penalized (Table IV).

Setting $P = 0$ lowers held-out safe success by 3.33 percentage points, and the count penalty lowers it by 2.58 points. Both intervals exclude zero, also when the patch-proposal and structured-output families are left out. These two output-format families cause most of the variation between seeds. Removing the task envelope raises safe success by 6.00 points, but only through these two families, and it is 1.19 points below the full reward on the others. Over-privileged success is close to zero for every variant on these held-out tasks, so the differences come from task completion. The privilege penalty therefore improves internal safe success, and the graded penalty performs better than the count penalty.

D. External Evaluation by Independent Benchmarks

Table V shows results on two independent benchmarks that were not used to select the training checkpoint. MetaTool measures general tool-selection ability and serves as a proxy for tool capability retention. On the other hand, FORTIS measures over-privileged action choice among items through parsing policy outputs. These tasks are adjacent in concept, but functionally out-of-distribution, and significantly greater improvements can likely be made using our proposed privilege-aware framework through training on similar tasks.

On MetaTool, Seed 1 answers 836 of 1,000 items correctly, compared with 819 for base. This result shows that internal improvements in safe success and reduction in over-privileged action do not degrade general tool-selection ability, and the gain is statistically significant ($p = 4.55 \times 10^{-4}$).

FORTIS Task 1 gains a 2 percentage-point improvement in exact minimum skill, and a 1.67-point reduction in over-privilege for the trained policy compared to the base policy, with the total safe rate improvement being statistically significant at $p = 0.026$. FORTIS Task 2 is affected by differences in output validity, as only 1,250/1,543 items produced parseable outputs for both policies, with the base policy leaving substantially more items empty or unparseable. Hence, the policies can only be compared on their shared parseable outputs. On the 1,250 items parsed by every evaluated policy,

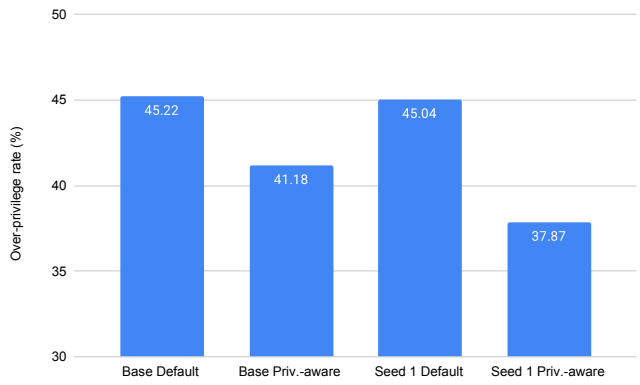


Fig. 3. ToolPrivBench over-privilege under the default and privilege-aware prompts. The vertical axis begins at 30%.

over-privilege falls from 43.28% for base to 40.72% for Seed 1 ($p = 2.11 \times 10^{-5}$). This suggests that the trained policy makes better authority choices on comparable tasks.

E. Prompt and Policy Effects on ToolPrivBench

ToolPrivBench tests escalation between substitutable standard and higher-authority tools after transient failures. This differs from the internal environment, where terminal and MCP actions may be complementary and where a higher-authority action can be required by the task.

Under the default ToolPrivBench prompt, the over-privilege rate is 45.2% for base and 45.0% for Seed 1, and a paired test finds no statistically significant difference. The overall rates are nearly equal, but the categories differ. Seed 1 has lower over-privilege than base in data over-exposure, scope expansion, and temporal persistence, and higher over-privilege in authority escalation and safety bypass.

There is a difference, however, under prompting conditions (Fig. 3). The privilege-aware and task-specifying instruction prompt lowers the base rate to 41.2% and the Seed 1 rate to 37.9%. The prompt therefore reduces over-privilege by 4.0 points for base and by 7.2 points for Seed 1. The prompted Seed 1 condition is 7.4 points below the default base condition.

F. Continuation Study Results

The original 1,500-step Seed 1 run performs well with the internal validation set, but it does not have as marked of an improvement in the external benchmarks. This is especially true of the ToolPrivBench failure escalation subsection. We therefore ran a 400-step continuation after the original training using ToolPrivBench-informed task schemas to examine the ability of the framework to apply to different tasks and domains. As the original framework was proposed and created before the release of the ToolPrivBench paper, this serves as a case study of the framework's ability to generalize to least-privilege aware training scenarios in separate tasks and domain types.

We created 200 corrective tasks styled after ToolPrivBench tasks and mixed in 200 tasks from the original 1,500 training tasks as rehearsal against catastrophic forgetting [22]. The corrective tasks cover transient retries on failure, bounded retries, convenience traps, escalation-necessary tasks, and equal-authority alternatives. No ToolPrivBench task was used as a sample, and the 200 corrective tasks were checked for direct overlap with ToolPrivBench using keyword matching, so the continuation does not train on the evaluation set. The broker, six-dimensional risk vector calculation, envelope and schema authoring setup, verifiers, and scalar reward remain unchanged. Only the task styling and families change.

Table VI compares the parent Seed 1 policy with the continuation policy. We monitored a held-out validation set of 50 tasks drawn from the same 400-task mix alongside the standard internal benchmark. On this 50-task set, safe success increases from 67.75% to 90.50%. On the internal 500-task validation set, the results do not significantly change, with safe success rising by 8 episodes, or 0.28 percentage points. MetaTool and FORTIS also show no statistically significant change in overall results, indicating capability retention.

On ToolPrivBench, the target of the continuation, default-prompt over-privilege falls from 245/544 to 207/544 scenarios, or from 45.04% to 38.05%, a statistically significant decrease of 6.99 percentage points ($p = 3.7 \times 10^{-5}$). Benchmark success also rises from 514 to 533 scenarios, so the decrease is not simply due to the model doing less.

The training families targeted by the corrective study also showed greater improvement. For example, the convenience-trap safe success rises from 8/64 to a full 64/64 and transient retry rises from 121/152 to 142/152. This shows the success of the continuation in fixing the behavior it was designed for, and we believe it also shows the ability of the framework to generalize for training outside our initial training tasks.

Without prompting, the continued policy's over-privilege of 38.05% is lower than that of the prompted base model at 41.18% and close to prompted Seed 1 at 37.87%. The privilege-aware prompt also has its largest effect on the continued policy, lowering over-privilege from 38.05% to 26.65%, an 11.40 percentage-point decrease, while benchmark success changes only from 533 to 532 scenarios.

We also ran the continuation with the full reward, $P = 0$, and the count penalty, using five seeds each. Mean ToolPrivBench over-privilege is 36.69% for the full reward, 42.21% for $P = 0$, and 35.55% for the count penalty. $P = 0$ is significantly worse than the full reward (seed-level permutation test, $p = 0.040$), and the full reward and the count penalty are not significantly different. The privilege penalty therefore accounts for part of the continuation's ToolPrivBench improvement.

G. Interpretation and Discussion

The base and trained policies both received the same least-privilege rules and prompt, along with the same environment. The difference of 34.12 points in safe success is therefore at least partially a gap in the ability to apply and understand these least-privilege rules. Furthermore, the prompt ablation results suggest that the trained policy conducts least-privilege

TABLE VI
SEED 1 PARENT VS. 400-TASK CORRECTIVE CONTINUATION

Evaluation	Seed 1 Parent	Continuation Candidate	Change
50-task validation set	271/400 (67.75%)	362/400 (90.50%)	+22.75 pp
500-task safe success	2,852/2,896 (98.48%)	2,860/2,896 (98.76%)	+0.28 pp
MetaTool accuracy	836/1,000 (83.60%)	835/1,000 (83.50%)	-0.10 pp
FORTIS Task 1 exact match	237/600 (39.50%)	238/600 (39.67%)	+0.17 pp
FORTIS Task 1 over-privilege	297/600 (49.50%)	293/600 (48.83%)	-0.67 pp
FORTIS Task 2 exact match	308/1,543 (19.96%)	321/1,543 (20.80%)	+0.84 pp
ToolPrivBench over-privilege	245/544 (45.04%)	207/544 (38.05%)	-6.99 pp
ToolPrivBench privilege-aware prompt	206/544 (37.87%)	145/544 (26.65%)	-11.21 pp

action regardless of the prompt, at least in a terminal and MCP interface similar to what it is trained on, and that this behavior is internalized by the model, not triggered only by prompting.

We focus on safe success for internal training and benchmarking because it requires both task completion and the absence of an excess-authority event. A security metric that tests only over-privilege can be improved simply by making the agent inactive or having it refuse. Such behavior may prevent over-privilege in some applications, but it is self-defeating because it degrades capability, and it cannot easily be combined with reinforcement learning aimed at improving task ability.

Unlike MetaTool and FORTIS, ToolPrivBench shows no difference between policies under its default prompt. ToolPrivBench mainly tests whether an agent retries a lower-authority tool or switches to a higher-authority tool after a transient failure. The continuation tasks train this decision directly, and the continuation lowers default-prompt over-privilege from 45.04% to 38.05%.

We believe the large drop under privilege-aware prompting, in the original Seed 1 policy and especially in the continued policy, shows that the model has learned how to act with least privilege in these task families, but needs prompting to know how to apply this to an unfamiliar interface and what exactly is expected. The practical takeaway for deployment is that post-training changes a policy's default choices and teaches behavior, while explicit rules and specifications still help that behavior appear in new settings.

The results also show that the learned policy should still sit behind ordinary system controls. This includes system prompting relevant to the deployed system and environment, such as what to do when encountering failures and the agent's role and expected authority. Privilege-gating systems like Claude Code auto mode are also still useful as it is best practice to keep sensitive operations behind separate, smaller trusted interfaces [23]. Our training can reduce how often models ask for unnecessary authority, and improve how well models can follow least-privilege instructions. However, it cannot give enforcement guarantees or deterministic fail-safes that come with such systems as permission gates.

V. LIMITATIONS AND FUTURE WORK

The tasks are largely synthetically generated from structured families with clear, predefined grouping and testing patterns. While whole-family holdouts and template-group checks reduce construction and cross-family dependence, they do not establish performance on unrestricted production repositories or services. The authority dimensions, weights, and sufficient envelopes encode judgments that may not be completely correct, though we have taken measures to reduce this by having a detailed risk archive justification section for each task. Furthermore, the study uses only one 4B model and one adapter family, though previous work suggests representation-level similarities across large language model families and scales that may let post-training transfer [24].

The weak ToolPrivBench improvement of Seed 1, together with the early saturation of internal gains by step 500, suggests that the gap is at least partly due to insufficient task variety in the catalog. Reinforcement learning is known to mainly reweight behaviors the model can already produce rather than create new ones [25]. On a benchmark that supplies no cues for what to do and does not share the structure of the training environment, the policy stays closer to base behavior, which is what we observe.

For future work, we hope to further explain the prompt-specific gap between the base and trained policies on ToolPrivBench. A wider training run with more task variety, including coverage of tasks similar to external benchmarks such as FORTIS and ToolPrivBench, would further measure the generalizability of this framework. Ablations that remove one risk dimension at a time would show the contribution of each dimension of the risk vector.

The continuation starts from the selected Seed 1 rather than from the base model. It was also post hoc in its motivation, so it is not a completely unbiased demonstration of generalization. A useful future experiment would mix the continuation tasks into training from the base model.

Training may also be unstable without cold-start measures such as supervised fine-tuning (SFT), as Seeds 0 and 2 often formatted the arguments of their first call to an unfamiliar MCP tool as XML instead of JSON, which the broker rejects as malformed. In a 50-task diagnostic, Seeds 0 and 2 completed 0/80 and 55/80 unseen-MCP episodes, compared with 80/80 for Seed 1. A small SFT cold start would likely stabilize training and reduce the need for multiple seeds [26]. ToolPrivBench, for example, uses cold-start SFT to train its Qwen3 family models [7]. We chose multiple seeds with pure reinforcement learning because of Qwen3.5's strong existing tool-use capability.

Finally, the reward weights were selected manually rather than learned. The reward ablation removes or changes the privilege term but does not tune the weights, so better weight combinations may exist. We also hope future work explores on-policy or otherwise automatic authoring of task authority envelopes, toward a more scalable least-privilege learning loop.

VI. Conclusion

Minimally sufficient least privilege can be learned in an executable agent environment and across multi-tool actions without degrading capability. A least-privilege-aware post-training framework combining a six-dimensional risk vector with pre- and post-action audits, predefined sufficient-authority envelopes, evidence gates, and exact-state verifiers raises safe success from 64.36% to 98.48% on the complete 500-task comparison and reduces successful excess-authority events from 4.56% to 0.79%, and the reward ablation shows that removing the privilege penalty lowers safe success. Most of the internal gain appears by step 500, suggesting that more task variety would bring further gains. Independent tests show retained tool competence and improved results under explicit privilege-aware prompting, while failure-conditioned escalation remains an occasional unresolved policy error. The continuation shows the framework's generality and modularity. Using the same broker, task-envelope authoring standards, verifiers, and excess-privilege calculation, it raises safe success from 67.75% to 90.50% on its 50-task validation set and reduces ToolPrivBench over-privilege from 45.04% to 38.05%, and further under the privilege-aware prompt, while maintaining capability on the earlier benchmarks. The framework is therefore useful as a reusable and modular measurement and training interface for least-privilege-aware training. As the trained policy can still encounter unfamiliar schemas, new scenarios, and otherwise out-of-distribution tasks, this framework of least-privilege-aware post-training should be used as a complement rather than a replacement for other permission gating methods, like invocation control systems.

Code and Model Availability

The code for training, internal evaluation, and the continuation study, together with the task catalog, is available at https://github.com/Astrowebdeving/least_privilege_grpo. The trained model weights and checkpoint adapters are available at https://huggingface.co/tu101/least-privilege-model.

Acknowledgment

Generative AI tools, specifically the large language models ChatGPT and Claude, were used in proof-reading, verification, and editing throughout the paper's text. The majority of the repository's code and task authoring was synthetic, generated by the tools of Codex and Claude Code with human supervision. The authors take full responsibility for the results and work of this paper.

References

[1] S. Zhuang and D. Hadfield-Menell, "Consequences of misaligned AI," in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 15 763–15 773.

[2] J. H. Saltzer and M. D. Schroeder, "The protection of information in computer systems," *Proc. IEEE*, vol. 63, no. 9, pp. 1278–1308, Sep. 1975.

[3] Z. Ji, Z. Li, P. Ma, W. Jiang, Y. Gao, and S. Wang, "Measuring the permission gate: A stress-test evaluation of Claude Code's auto mode," arXiv:2604.04978, 2026.

[4] Z. Yan *et al.*, "Do coding agents understand least-privilege authorization?" arXiv:2605.14859, 2026.

[5] J. Hughes, "How we built Claude Code auto mode: A safer way to skip permissions," Anthropic Engineering, Mar. 25, 2026. [Online]. Available: https://www.anthropic.com/engineering/claude-code-auto-mode

[6] S. Li *et al.*, "FORTIS: Benchmarking over-privilege in agent skills," arXiv:2605.09163, 2026.

[7] K. Yang *et al.*, "When lower privileges suffice: Investigating over-privileged tool selection in LLM agents," arXiv:2606.20023, 2026.

[8] Y. Huang *et al.*, "MetaTool benchmark for large language models: Deciding whether to use tools and which to use," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2024.

[9] X. Wang *et al.*, "OpenHands: An open platform for AI software developers as generalist agents," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2025.

[10] J. Yang *et al.*, "SWE-agent: Agent-computer interfaces enable automated software engineering," in *Advances in Neural Information Processing Systems*, vol. 37, 2024.

[11] Model Context Protocol, "Tools," Model Context Protocol Specification, protocol revision 2025-11-25, 2025. [Online]. Available: https://modelcontextprotocol.io/specification/2025-11-25/server/tools

[12] Z. Shao *et al.*, "DeepSeekMath: Pushing the limits of mathematical reasoning in open language models," arXiv:2402.03300, 2024.

[13] Z. Liu *et al.*, "Understanding R1-Zero-like training: A critical perspective," in *Proc. 2nd AI for Math Workshop at ICML*, 2025, arXiv:2503.20783.

[14] L. Fluri, L. Lang, A. Abate, P. Forré, D. Krueger, and J. M. V. Skalse, "The perils of optimizing learned reward functions: Low training error does not guarantee low regret," in *Proc. 42nd Int. Conf. Mach. Learn. (ICML)*, ser. PMLR, vol. 267, 2025, pp. 17 306–17 377.

[15] A. Y. Ng, D. Harada, and S. Russell, "Policy invariance under reward transformations: Theory and application to reward shaping," in *Proc. 16th Int. Conf. Mach. Learn. (ICML)*, 1999, pp. 278–287.

[16] Qwen Team, "Qwen3.5-4B model card," 2026. [Online]. Available: https://huggingface.co/Qwen/Qwen3.5-4B

[17] J. Schulman, "LoRA without regret," Thinking Machines Lab, Sep. 29, 2025. [Online]. Available: https://thinkingmachines.ai/blog/lora/

[18] L. von Werra *et al.*, "TRL: Transformers reinforcement learning," Hugging Face, software, version 1.5.1, 2026. [Online]. Available: https://github.com/huggingface/trl/tree/v1.5.1

[19] W. Kwon *et al.*, "Efficient memory management for large language model serving with PagedAttention," in *Proc. 29th ACM Symp. Operating Systems Principles (SOSP)*, 2023, pp. 611–626.

[20] Q. McNemar, "Note on the sampling error of the difference between correlated proportions or percentages," *Psychometrika*, vol. 12, no. 2, pp. 153–157, 1947.

[21] K. Lyu, H. Zhao, X. Gu, D. Yu, A. Goyal, and S. Arora, "Keeping LLMs aligned after fine-tuning: The crucial role of prompt templates," in *Advances in Neural Information Processing Systems*, vol. 37, 2024, arXiv:2402.18540.

[22] A. Robins, "Catastrophic forgetting, rehearsal and pseudorehearsal," *Connection Science*, vol. 7, no. 2, pp. 123–146, 1995.

[23] N. Provos, M. Friedl, and P. Honeyman, "Preventing privilege escalation," in *Proc. 12th USENIX Security Symp.*, 2003.

[24] Y. Huang, C. Huang, D. Feng, W. Lei, and J. Lv, "Cross-model transferability among large language models on the platonic representations of concepts," in *Proc. 63rd Annu. Meeting Assoc. Comput. Linguistics (ACL)*, Jul. 2025, pp. 3686–3704.

[25] Y. Yue *et al.*, "Does reinforcement learning really incentivize reasoning capacity in LLMs beyond the base model?" arXiv:2504.13837, 2025.

[26] D. Guo *et al.*, "DeepSeek-R1 incentivizes reasoning in LLMs through reinforcement learning," *Nature*, vol. 645, no. 8081, pp. 633–638, Sep. 2025.